Gather

Software Design Description (SDD)

Section 6 - Architecture, Detailed Design and Database

CMSI 4071 Senior Project I

Loyola Marymount University

Daniel Ruskin, Aristotle Kaporis, Luca Piccinini, Ivana Krajina, Greg Forston

November 4, 2026

6.1 Introduction

This document has the architecture and detailed design for our senior project, Gather. Gather is a web app for students that puts all their assignments, exams and deadlines in one place. Tasks can be added by hand, or pulled in from Brightspace, PDFs (like a syllabus) and class web pages. Anything that gets pulled in from outside has to be reviewed and approved by the student before it actually shows up in their task list and calendar.

Some of the stuff in here is already built and some of it is still planned. We tried to say which is which in each section, since we're still in the middle of building it.

6.1.1 System Objectives

The main goal of Gather is to give students one dashboard and calendar where they can see everything that's due, sorted by date, and check things off when they're done. Right now students have to look at Brightspace, the syllabus, and sometimes a professor's own website to figure out what's due, and it's easy to miss something.

The objectives of the app are:

- Show one dashboard and calendar with all assignments sorted by due date, and let the student mark them complete (SRS 5.3.1).
- Let students add, edit and delete tasks manually, and save them to their own account (SRS 5.3.1, 5.3.2).
- Connect to Brightspace (with the student's permission) and get class deadlines from it. Also accept PDFs and web page links as other ways to get assignments in (SRS 5.3.2).
- Use Llama 3 through OpenRouter to read the text from those sources and find assignment info. The results are saved as "proposed" and the student has to approve them before they become real tasks (SRS 5.2.1, 5.3.2).
- Keep user data safe, including only letting users see their own data, encrypting Brightspace tokens, and logging every time we use the Brightspace API (SRS 5.3.3).
- Be reasonably fast: dashboard loads in under 5 seconds, task updates in under 3 seconds, and imports show up for review within 10 seconds after processing finishes (SRS 5.4).

6.1.2 Hardware, Software, and Human Interfaces

6.1.2.1 User Interface (GUI)

Students use Gather through a web browser. The React frontend we have right now has a sidebar on the left with Dashboard, My Tasks, Review Imports and Sources. The dashboard shows task counts (total, to do, completed), an Add Task button, the list of assignments, the pending imports with approve/reject buttons, and a calendar that slides out. Users interact with it using a mouse/trackpad or touch and the keyboard.

We still need to make the sign in, sign up, password reset, settings, upload and source management pages. Those are in the SRS (5.3.1) but they aren't in App.jsx yet.

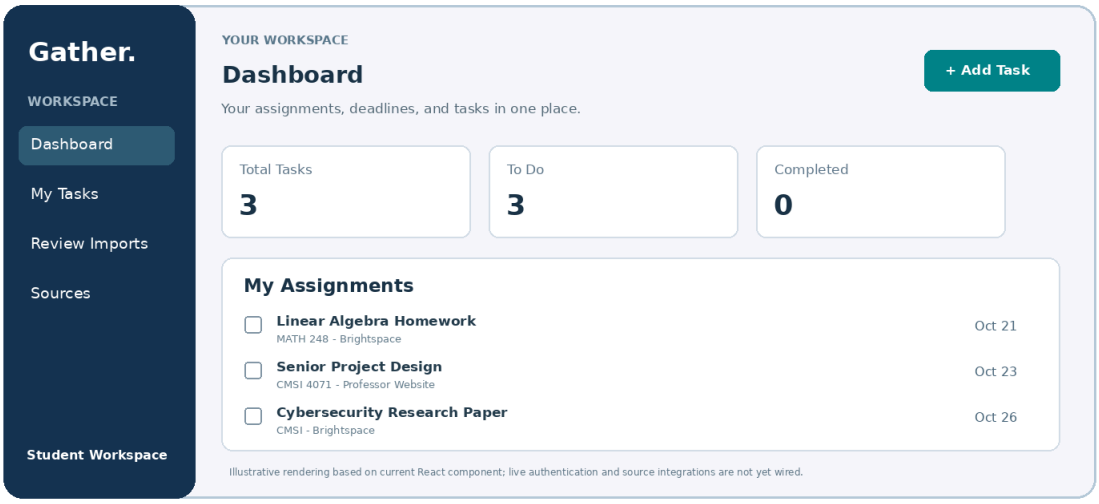


Figure 6-1. Mockup of the Gather dashboard (based on our current React layout, not connected to the backend yet)

6.1.2.2 Hardware Interfaces

Gather doesn't need any special hardware. A student just needs a laptop, desktop, tablet or phone with a screen, a keyboard and/or mouse or touchscreen, and an internet connection. The app runs on Firebase Cloud Functions, so we don't run our own server. For development we use Windows, macOS and Linux.

6.1.2.3 Network / Communication Interface

The browser talks to our backend over HTTPS. Normal API requests and responses are JSON (application/json). Routes that need a logged in user also send the Firebase ID token in an `Authorization: Bearer <token>` header. PDF uploads will be sent as multipart/form-data and URL imports as JSON. When the backend trades OAuth codes for tokens with Brightspace it uses URL-encoded form bodies, which is what Brightspace expects.

Right now index.js only has `express.json()` set up. We still need to add file upload handling and a CORS policy.

6.1.2.4 React

Version 19.2.8 (react and react-dom). We use React for the whole frontend, which is in frontend/src/App.jsx. Right now the task list is stored in React state (useState) with sample data, it isn't loading from the backend yet.

6.1.2.5 Vite

Version 8.3.0, with @vitejs/plugin-react 6.1.1. Used for running the frontend locally and building it. It only affects the frontend.

6.1.2.6 Node.js and Express

Node.js 24 (set in Backend/functions/package.json) and Express 4.21.2. Express handles our API routes and middleware. Right now the backend has three route groups: /assignments, /users and /brightspace.

6.1.2.7 Firebase Cloud Functions

firebase-functions version 7.0.0. This is how our Express app actually gets deployed. The whole API is exported as one HTTPS function called `api`.

6.1.2.8 Firebase Admin SDK, Authentication and Firestore

firebase-admin version 13.6.0. The backend uses this to check Firebase ID tokens and to read and write Firestore. Firebase Authentication is how users log in. Firestore is our database and it stores users, assignments, Brightspace OAuth data and Brightspace API usage logs. We still need to add the Firebase Auth SDK to the frontend so users can actually log in and get a token, and we still need to pick which login providers to support.

6.1.2.9 cors

Version 2.8.5. It's in package.json but we haven't actually added it to index.js yet. We'll use it to only allow requests from our own frontend domain.

6.1.2.10 Brightspace (D2L) API

Gather connects to Brightspace using OAuth 2.0 Authorization Code flow with PKCE (S256). LMU ITS requires PKCE for any app that connects to Brightspace, so we can't use a simpler login flow. The server builds the authorize, token and revoke URLs from BRIGHTSPACE_BASE_URL. The other environment variables we need are BRIGHTSPACE_CLIENT_ID, BRIGHTSPACE_CLIENT_SECRET, BRIGHTSPACE_REDIRECT_URI and BRIGHTSPACE_SCOPES. The connect, callback, status and disconnect endpoints are done. Getting the list of courses and the actual assignments from Brightspace is still to do.

LMU ITS requires us to keep a record of every time Gather uses the Brightspace API. So every call we make to Brightspace (token exchange, token refresh, revoke, and later getting courses and assignments) goes through one wrapper function that also writes a usage log entry. This is described more in 6.3.1.10 and 6.3.3.5.

6.1.2.11 OpenRouter / Llama 3

We picked OpenRouter to call the LLM and Llama 3 as the model (from the SRS). The backend will send the cleaned up text from a PDF or web page over HTTPS with our API key (kept on the server), and ask for the assignments back as JSON. We'll check the JSON against a schema and handle rate limits and retries. We haven't locked in the exact Llama 3 model version yet, we're going to test a couple first.

6.1.2.12 PDF and Web Page Parsing

The ingestion part will take PDFs up to 5 MB and http/https links, pull out the text, and send it to the AI part. Uploaded PDFs will go into Firebase Cloud Storage temporarily and get deleted once the text is extracted (from the SRS). We haven't added a PDF parsing library or a web scraping library yet, so those versions are TBD.

6.1.2.13 @google-cloud/kms and Node crypto

@google-cloud/kms version 6.3.0 is in package.json in case we want to use Google's key management later, but right now we don't call it. Instead our crypto helper uses Node's built in crypto module to do PKCE and to encrypt Brightspace refresh tokens with AES-256-GCM, using a key from the TOKEN_ENCRYPTION_KEY environment variable. Content from Brightspace and professor websites is treated as untrusted data.

6.2 Architectural Design

We split Gather into parts based on what they do: the frontend, the API, the Brightspace connection, getting text out of PDFs/websites, the AI extraction and review, and the database. We did this so the PDF stuff, the Brightspace stuff and the LLM stuff aren't all mixed together in one big backend file. The diagrams in this section are high level, the details are in 6.3.

6.2.1 Major Software Components

Component (CSC)	What it does	Status
CSC-1: Web Client (GUI)	React pages, sorting/filtering tasks, calendar, manual task entry, approving/rejecting imports. Runs in the browser.	Prototype done, login and API connection still to do
CSC-2: API / Identity	Express routes, checking tokens, validating requests, assignment and user handlers.	Main routes done, not hooked up to UI yet
CSC-3: Brightspace Integration	PKCE OAuth (required by LMU ITS), encrypting and storing tokens, disconnecting, and later syncing courses and assignments.	OAuth done, syncing planned
CSC-4: Data Ingestion	PDF upload, fetching URLs, cleaning up text, keeping track of where text came from, temp files.	Planned
CSC-5: AI Extraction and Review	Calling Llama 3, checking the JSON, cleaning up results, checking for duplicates, student approval.	Planned (only a fake review demo in the UI)
CSC-6: Data Management	Firestore user profiles, assignments, tokens, Brightspace usage logs, and later import/course records.	Partly done

How these match up to the SRS: CSC-1 is mostly SRS 5.3.1. CSC-2 and CSC-3 cover 5.3.2.1 through 5.3.2.11. CSC-4 and CSC-5 cover 5.3.2.12 through 5.3.2.18. CSC-6 is 5.3.3. (Note: the

SRS accidentally has two requirements numbered 5.3.1.17. One is the add assignment form and one is the edit dialog.)

6.2.2 Major Software Interactions

Browser to API. When the student does something, the React app sends an HTTPS request with JSON and their Firebase ID token. Express checks the token and then passes the request to the right handler. This is designed but the demo frontend doesn't call the API yet.

API to Firestore. The handlers call helper functions that use the Firebase Admin SDK. Everything is stored under users/{uid} so each user's assignments, profile and Brightspace info are kept separate.

Brightspace connection. This uses OAuth 2.0 with PKCE, which LMU ITS requires. The student gets sent to Brightspace to approve access. Brightspace sends them back to our callback with a code and a state value. The server checks the state matches what we saved, trades the code for tokens over HTTPS, and saves the refresh token encrypted in Firestore.

Import to AI. A PDF or URL gets turned into text (and we remember where it came from). That text goes to the AI service, which sends back proposed assignments as JSON. These get saved as candidates and do NOT become real tasks automatically.

Review to database. If the student approves a candidate it becomes a normal assignment under their account. If they reject it, it gets marked rejected. We check for duplicates so the same thing doesn't get imported twice.

Errors. If the token is bad the request stops before any other code runs. If a source or the AI fails, the student sees an error and their existing tasks aren't touched. Disconnecting Brightspace revokes the token (if it can) and deletes the saved connection data.

Brightspace usage logging. Any time the backend calls the Brightspace API, the Brightspace connector writes a log entry to Firestore saying who it was for, what was called, and if it worked. This is required by LMU ITS. If writing the log fails, the Brightspace call still goes through, but the error gets printed to the Cloud Functions logs so we can see it.

6.2.3 Architectural Design Diagrams

Below are three diagrams: a use case diagram, a top level class diagram and a component/deployment diagram. More detailed versions are in 6.3.4.

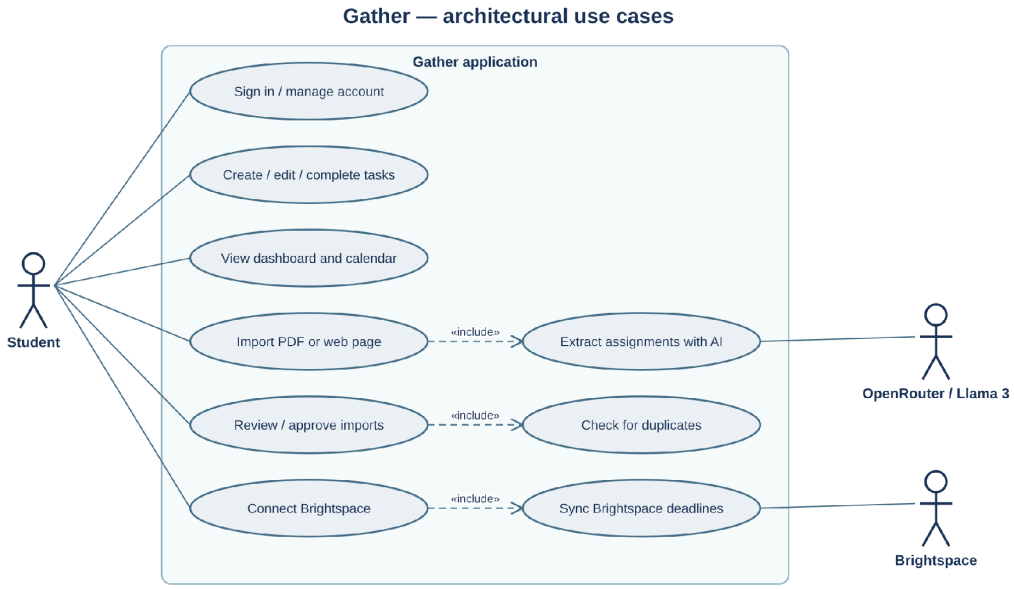


Figure 6-2. Use case diagram

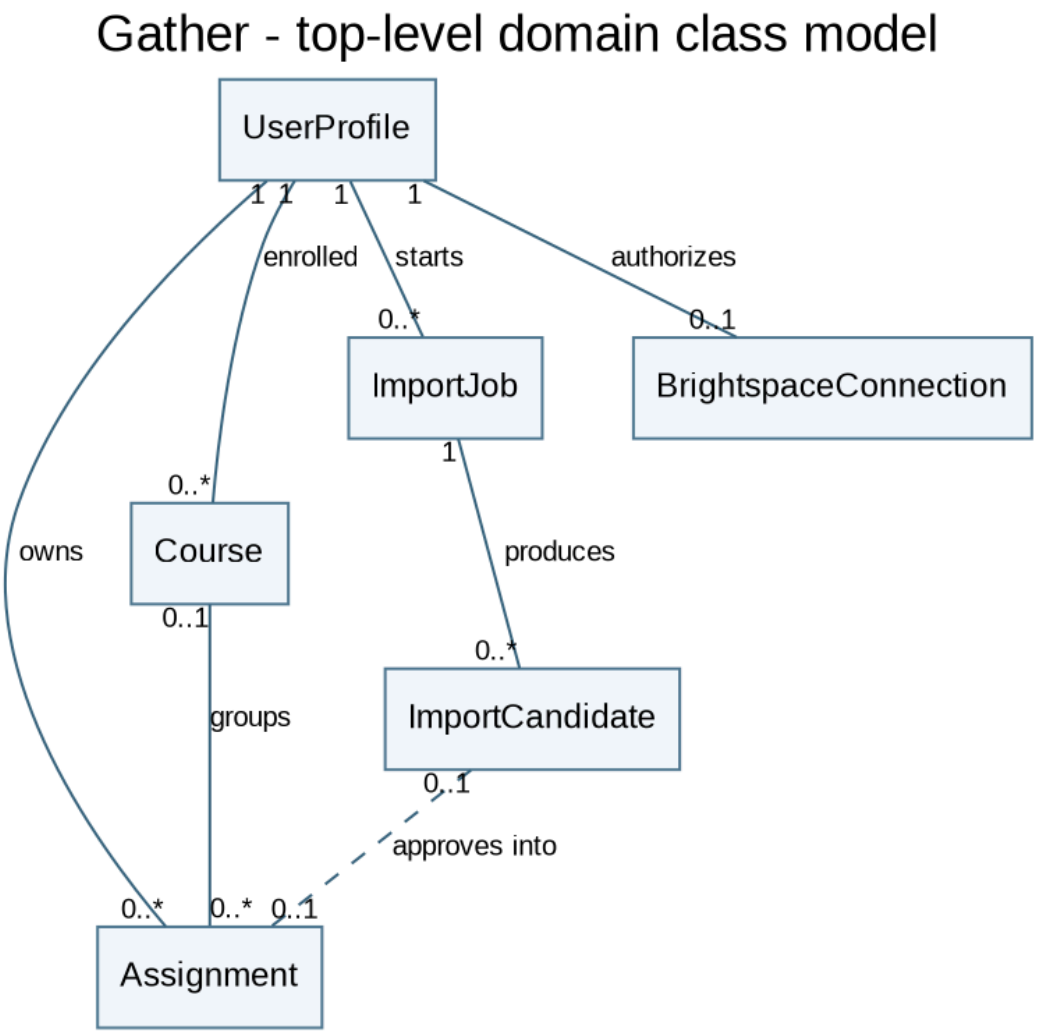


Figure 6-3. Top level class diagram

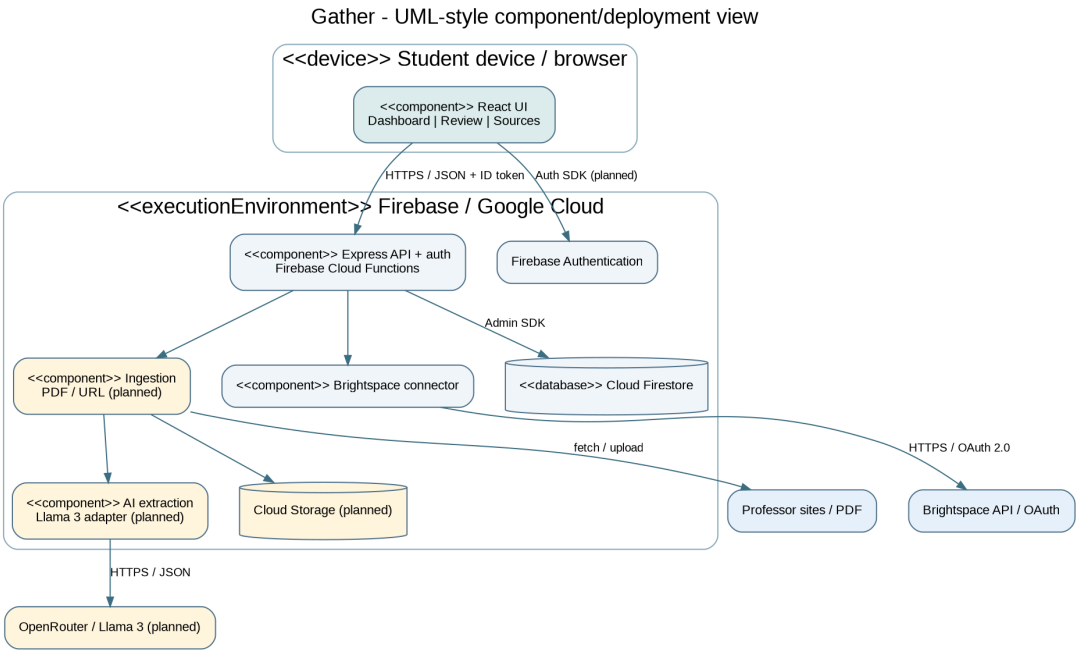


Figure 6-4. Component / deployment diagram

6.3 CSC and CSU Descriptions

The whole Gather app is our CSCI. It's made of the six CSCs from 6.2.1, and each CSC is made of CSUs, which are described below. One thing to note is that most of our code is JavaScript functions and a React function component, not actual classes. So the "classes" below are more like the units we're designing (what data they hold and what operations they have) and some of them are planned and don't exist in the code yet.

Which CSUs go in which CSC:

CSC	CSUs
CSC-1 Web Client	6.3.1.6 App Workspace Controller
CSC-2 API / Identity	6.3.1.7 AuthGuard, 6.3.1.8 AssignmentService, 6.3.1.9 UserService
CSC-3 Brightspace Integration	6.3.1.10 BrightspaceConnector
CSC-4 Data Ingestion	6.3.1.11 IngestionService
CSC-5 AI Extraction and Review	6.3.1.12 AIExtractionAndReviewService
CSC-6 Data Management	6.3.1.1 Assignment, 6.3.1.2 UserProfile, 6.3.1.3 Course, 6.3.1.4 ImportCandidate, 6.3.1.5 ImportJob

6.3.1 Class Descriptions

The following sections give the details of all the classes in Gather. We start with the smaller data classes and then go to the bigger service classes. Types like Timestamp, string and array mean the Firestore/API type.

6.3.1.1 Assignment

Status: data shape is done in Backend/functions/src/helpers/assignments.js, methods are planned. This is one assignment/deadline that belongs to one user. It's the main thing the app is about.

Fields:

- id : string - Firestore document ID
- userId : string - the owner's uid. This comes from the token, the browser can't set it
- title : string - name of the assignment, can't be empty
- courseId : string (optional) - which course it's for
- dueAt : Timestamp (optional) - when it's due
- type : string - assignment, exam, etc. Default is "assignment"
- source / sourceUrl : string (optional) - where it came from and a link if there is one
- status : string - active or completed (see 6.3.3.4, the current code still uses "pending" as the default)
- confidence : number (optional) - how sure the AI was, only for imported tasks

- createdAt / updatedAt : Timestamp - set by the server

Methods:

- validate() : ValidationResult (planned) - rejects missing title, bad dates, bad status, or trying to change the owner
- markComplete() (planned) - changes status to completed, keeps the owner and timestamps

6.3.1.2 UserProfile

Status: done, schema is in helpers/users.js.
Stores basic info about the student. This is just the data, the operations on it are in UserService (6.3.1.9).

Fields:

- uid : string - Firebase Auth user ID, also the document ID
- displayName : string (optional) - the user's name
- email : string (optional) - from their login
- brightspaceIcsUrl : string (optional) - Brightspace calendar link if they saved one
- createdAt / updatedAt : Timestamp

Methods: none (see UserService).

6.3.1.3 Course

Status: planned (SRS 5.3.3.7).
A class the student is in, either from Brightspace or added manually.

Fields:

- id : string - our internal course ID
- userId : string - owner's uid
- name : string - course name
- term : string (optional) - semester
- externalCourseId : string (optional) - the Brightspace course ID
- source : string - "brightspace" or "manual"

Methods:

- findByExternalId(uid, externalId) (planned) - looks up a course so we don't make duplicates

6.3.1.4 ImportCandidate

Status: planned. Right now there are only fake imports hardcoded in App.jsx.
An assignment that the AI found but the student hasn't approved yet.

Fields:

- id / jobId : string - the candidate's ID and the import job it came from
- title : string - suggested name
- dueAt : string (optional) - suggested due date as ISO 8601
- courseId : string (optional) - suggested course
- confidence : number (optional) - how confident the AI was
- evidence / sourceUrl : string (optional) - the bit of text it came from, so the student can check it
- decision : string - proposed, approved, rejected or duplicate
- assignmentId : string (optional) - the assignment that got created when it was approved

Methods:

- validateCandidate() (planned) - checks title, date, confidence and decision
- edit(changes) (planned) - lets the student fix fields before approving
- approve() (planned) - creates a real Assignment after checking for duplicates
- reject() (planned) - marks it rejected, no assignment is created

6.3.1.5 ImportJob

Status: planned.
Keeps track of one import (a PDF, URL or Brightspace course) from when it's submitted to when it's ready to review.

Fields:

- id / userId : string - job ID and owner
- sourceType : enum - pdf, url or brightspace
- sourceRef : string (optional) - the temp storage file or the URL
- status : enum - queued, extracting, parsing, ready, failed
- errorCode : string (optional) - why it failed
- createdAt / updatedAt : Timestamp

Methods:

- start(source) (planned) - checks the source and creates a queued job
- recordCandidates(items) (planned) - saves the candidates and sets status to ready
- fail(errorCode) (planned) - marks the job failed without touching existing assignments

6.3.1.6 App Workspace Controller

Status: done, frontend/src/App.jsx (React function component).
This is basically the whole frontend right now. It controls the dashboard, task list, import review and the calendar.

Fields (state):

- tasks : array - the task list, right now sample data in useState, later from the database
- imports : array - pending imports, also sample data right now

- activePage : string - Dashboard, My Tasks, Review Imports or Sources
- calendarOpen : boolean - if the calendar drawer is open
- selectedDate : string (optional) - date picked on the calendar, used to filter tasks
- calendarMonth : Date - which month the calendar is showing
- sortedTasks / visibleTasks - tasks sorted by date and filtered (computed from tasks)

Methods:

- toggleTask(id) - checks/unchecks a task as completed
- addTask() - asks the user for title, course and date, checks them, and adds the task
- approveImport(item) - moves an import into tasks and takes it off the pending list
- rejectImport(id) - removes an import
- changeMonth(amount) - goes forward/back a month on the calendar and clears the selected date
- dateKey(day) - formats a day as YYYY-MM-DD
- formatDate(date) - formats a date for display (like "Oct 21")

6.3.1.7 AuthGuard

Status: done, Backend/functions/src/middleware/auth.js.
Middleware that blocks requests that don't have a valid login token.

Fields:

- req.headers.authorization : string - the "Bearer <token>" header from the client
- req.user : DecodedToken (optional) - the verified user, set after checking
- admin : FirebaseAdmin - the Firebase Admin instance

Methods:

- authenticate(req, res, next) - verifies the ID token, sets req.user, and calls next(). If the token is missing or bad it returns 401.

6.3.1.8 AssignmentService

Status: done (helper functions and handlers).
Handles creating, reading, updating and deleting assignments in Firestore.

Fields:

- db : Firestore - Admin SDK Firestore client
- uid : string (parameter) - the user ID from the verified token
- assignmentId : string (parameter) - the assignment document ID

Methods:

- getAssignmentsCollection(uid) : CollectionReference - returns users/{uid}/assignments
- createAssignment(uid, data) : Assignment - cleans up the fields, turns dueAt into a Timestamp, saves it

- getAssignments(uid) : Assignment[] - returns all of the user's assignments
- updateAssignment(uid, id, changes) : void - updates the assignment and updatedAt
- deleteAssignment(uid, id) : void - deletes the assignment

6.3.1.9 UserService

Status: done, helpers/users.js and handlers/users.js.
Saves and gets the student's profile.

Fields:

- db : Firestore
- uid : string - the logged in user
- profile : UserProfile - the users/{uid} document

Methods:

- syncUserProfile(user) : {isNew, profile} - creates the profile on first login, otherwise just returns it
- getUserProfile(uid) : UserProfile or null
- updateBrightspaceUrl(uid, url) : void - saves the Brightspace calendar URL

6.3.1.10 BrightspaceConnector

Status: OAuth part is done (helpers/brightspaceClient.js, helpers/brightspaceTokens.js, helpers/crypto.js, handlers/brightspace.js). Syncing courses/assignments is planned.
Handles connecting to Brightspace and keeping the tokens.

Fields:

- config : OAuthConfig - client ID, secret, base URL, redirect URI and scopes from environment variables
- codeVerifier / state : string - PKCE verifier and the state value for CSRF protection (temporary, one per login)
- encryptedRefreshToken : string - the AES-256-GCM encrypted refresh token, only on the server
- connectedAt / updatedAt : Timestamp
- usageLogCollection : CollectionReference - the brightspaceUsageLogs collection in Firestore

Methods:

- generateCodeVerifier() / generateCodeChallenge() / generateState() - make the PKCE and state values
- buildAuthorizeUrl(state, challenge) - makes the URL to send the user to Brightspace
- exchangeCodeForTokens(code, verifier) - trades the code for tokens
- savePendingAuth() / saveTokens() / getRefreshToken() - save and decrypt stuff in Firestore
- refreshAccessToken(token) - gets a new short lived access token

- revokeToken(token) / clearBrightspaceData(uid) - revoke the token and delete the saved data
- connectHandler / callbackHandler / statusHandler / disconnectHandler - the HTTP endpoints
- fetchEnrollments() / fetchAssignments() (planned) - get the student's courses and deadlines from Brightspace
- brightspaceRequest(uid, action, method, url, options) (planned) - the one function that actually sends requests to Brightspace. Every other method above that talks to Brightspace (exchangeCodeForTokens, refreshAccessToken, revokeToken, fetchEnrollments, fetchAssignments) calls this instead of calling fetch directly. It times the request and then calls trackBrightspaceUsage() whether the request worked or not. Doing it this way means nobody can forget to log a call.
- trackBrightspaceUsage(uid, action, method, endpoint, statusCode, durationMs) (planned) - record all instances of when the Brightspace API is used. This should be called anytime the Brightspace API is used (brightspaceRequest does this for us). This was a requirement given by LMU ITS for proper Brightspace API usage. It saves one BrightspaceUsageLog entry (6.3.3.5). The endpoint is saved without the query string so codes and tokens never end up in the log.
- getUsageLogs(uid, limit) (planned) - returns the most recent usage logs for one user, newest first. Used for debugging and if ITS asks for a report.

6.3.1.11 IngestionService

Status: planned (SRS 5.3.2.12-14, 18).
Turns PDFs and web pages into text that we can send to the AI.

Fields:

- maxPdfBytes : number - 5 MB limit
- allowedMimeTypes : string[] - PDF and HTML/text
- requestTimeoutMs : number - time limit for fetching URLs (we'll figure out the exact number when testing)
- sourceRef : string - the storage file or the URL
- extractedText : string - the cleaned up text that goes to the AI

Methods:

- validatePdf(file) - checks size and that it's actually a PDF
- validateUrl(url) - checks that the link is a real http/https URL
- extractPdfText(file) - gets text out of the PDF and remembers which page it came from
- extractWebText(url) - downloads the page and pulls out the assignment related text
- deleteTemporaryFile(sourceRef) - deletes the PDF from Cloud Storage after we're done
- submitImport(uid, source) : ImportJob - creates the import job and starts processing

6.3.1.12 AIExtractionAndReviewService

Status: planned (SRS 5.3.2.15-17).
Sends text to Llama 3, cleans up what comes back, and makes sure the student approves everything.

Fields:

- modelId : string - which Llama 3 model on OpenRouter (still picking)
- schemaVersion : string - version of the JSON format we expect back
- promptTemplate : string - the prompt we send. The text from outside is treated as untrusted
- candidates : ImportCandidate[] - the checked results
- dedupeKey : string - key made from source/course/title/due date (or Brightspace ID) to find duplicates

Methods:

- extractCandidates(sourceText) - calls the AI and parses the JSON
- validateSchema(candidate) - throws out bad output and bad dates
- detectDuplicates(uid, candidates) - compares with existing tasks and pending candidates
- saveProposed(uid, job, candidates) - saves them as proposed
- reviewCandidate(uid, id, decision, edits) - approve or reject, after checking the user owns it
- finalizeApproved(uid, ids) - creates the real assignments (safe to run twice, won't make duplicates)

6.3.2 Detailed Interface Descriptions

This section explains how the parts of the system send data to each other. Endpoints that are already in the code are marked done and the rest are planned.

6.3.2.1 Browser to API / Authentication

Everything goes over HTTPS with JSON. Protected routes need `Authorization: Bearer <Firebase ID token>`. The authenticate middleware calls `admin.auth().verifyIdToken()` and puts the user in req.user so handlers can use req.user.uid. If the token is missing or wrong it returns 401 with `{"error":"Unauthorized"}`. The frontend will convert Timestamps to ISO strings and show error messages. Upload handling and loading states still need to be added.

6.3.2.2 Assignment Endpoints (done)

All of these require AuthGuard. They all use POST right now (more RPC style than REST).

Endpoint	Body / Response
POST /assignments/create	Body: title (required), courseId, dueAt, type, source, sourceUrl, status,

Endpoint	Body / Response
	confidence. Returns 201 {message, id, assignment} or 400 if no title.
POST /assignments/read	No body. Returns 200 {assignments: [...]} for the logged in user.
POST /assignments/patch	Body {id, ...updates}. Returns 200 {message} or 400 if no id. The handler removes userId before updating.
POST /assignments/delete	Body {id} (or id in the query). Returns 200 {message} or 400 if missing.

Right now if Firestore fails these just return 500. We want to add better 400/404/409 errors and a list of which fields are allowed to be changed. The frontend doesn't call these yet.

6.3.2.3 User Profile Endpoints (done)

Endpoint	Body / Response
POST /users/login	Optional displayName/email. Creates or syncs the user from the token. 201 if new, 200 otherwise, returns {message, user}.
GET /users/profile	200 {user}, or 404 if not found.
POST /users/profile	Does the same thing as /users/login (we'll probably remove one of them).
POST /users/brightspace-url	Body {brightspaceIcsUrl}. Returns 200 with the saved URL.

6.3.2.4 Brightspace OAuth Endpoints (done)

Endpoint	Body / Response
POST /brightspace/connect	Needs login. Saves PKCE state/verifier and returns {authorizeUrl}. 409 if already connected.
GET /brightspace/callback	Public, Brightspace redirects here with code and state. Checks the state, trades the code for tokens, then redirects to a success or error page.
GET /brightspace/status	Needs login. Returns {connected: true/false} based on whether there's a saved token.
POST /brightspace/disconnect	Needs login. Revokes the token if possible, deletes the auth docs and any assignments that came from Brightspace.

This uses OAuth 2.0 Authorization Code with PKCE (S256) and a state value to stop CSRF. PKCE is required by LMU ITS for Brightspace apps. The redirect after callback always goes to APP_URL from the server config, not something the user passes in. Later calls to get courses/assignments will use refreshed access tokens. Every one of these outgoing calls to Brightspace goes through brightspaceRequest(), so each one gets a usage log entry.

6.3.2.5 Import, AI and Review Endpoints (planned)

Endpoint	Input / Output
POST /imports/pdf	multipart/form-data PDF (max 5 MB). Returns 202 {jobId, status: queued}. 413 if too big, 415 if not a PDF, 401 if not logged in.
POST /imports/url	JSON {url}. Returns 202 {jobId, status: queued}. 400 if it's not a valid URL.
GET /imports/:jobId	200 {jobId, status, candidates: [...], error}. 404 if not found or not yours.
POST /imports/:jobId/review	JSON {decisions: [{candidateId, decision, edits}]}. Returns 200 {approved, rejected, duplicates}.
GET /brightspace/courses	200 {courses: [{id, name, term}]}. Needs login and Brightspace connected.
POST /brightspace/import-course	JSON {courseId}. Returns 202 with an import job to review.

Inside the backend, ingestion will pass `{sourceType, sourceId, extractedText, sourceUrl}` to the AI service, and the AI service returns a JSON array of `{title, dueAt, courseId, details, evidence, confidence}`. We only save results that pass the schema check, and nothing gets activated automatically. If the AI call fails we return an error code and don't touch existing assignments.

6.3.2.6 Backend to Database

All database access goes through the Firestore Admin SDK, for example `db.collection("users").doc(uid).collection("assignments")` and the brightspaceAuth docs next to it. Since the Admin SDK skips Firestore security rules, our backend code has to check ownership itself. The planned import, course and candidate data will also be stored under users/{uid}. Brightspace usage logs go in a top level brightspaceUsageLogs collection with a uid field (see 6.3.3.5).

6.3.3 Detailed Data Structure Descriptions

6.3.3.1 Assignment

The assignment document in the backend looks like: {title, courseId, dueAt, type, source, sourceUrl, status, confidence, userId, createdAt, updatedAt}. title is required. dueAt is a Firestore Timestamp or null. confidence is a number or null. createdAt/updatedAt are server timestamps. The document ID gets returned as id. When sending dates to the frontend we use ISO 8601 strings with a timezone. We might add a separate dueDate (YYYY-MM-DD) field for assignments that only have a date, because otherwise timezones can make it show up a day off.

The React demo uses a different shape right now: {id, title, course, dueDate, source, completed}. We need to convert between them: course becomes a courseId, dueDate becomes dueAt, and completed becomes status = completed.

6.3.3.2 User and OAuth Data

User profile: {uid, displayName, email, brightspaceIcsUrl, createdAt, updatedAt}.

Brightspace pending doc: {state, codeVerifier, createdAt}. Brightspace tokens doc: {encryptedRefreshToken, connectedAt, updatedAt}. The encrypted token is stored as `ivHex:authTagHex:ciphertextHex`. The plain refresh token is never sent back to the frontend. The PKCE verifier and state should only work once and should expire (we still need to add the expiration check).

6.3.3.3 Import Data (planned)

- ImportJob: {id, userId, sourceType, sourceRef, status, errorCode, createdAt, updatedAt}
- Candidate: {id, jobId, title, details, courseId, dueAt, sourceUrl, evidence, confidence, decision, assignmentId, createdAt}
- SourceText: {sourceType, sourceRef, contentHash, extractedText, extractedAt, sourceUrl}

The parser should cut the text down to a reasonable length and keep the page number or URL so the student can see where an assignment came from when reviewing. When a candidate is approved we save the new assignment's ID on it so approving twice doesn't create two assignments.

6.3.3.4 Status Values and Errors

We're using set lists of values instead of random strings:

- Import job status: queued, extracting, parsing, ready, failed
- Candidate decision: proposed, approved, rejected, duplicate (approved but skipped because the task already exists)
- Assignment status: active, completed

The current helper code defaults assignment status to "pending" but the SRS (5.3.2.17) says imported items go from proposed to active. We're fixing this by keeping the proposed/approved/rejected stuff on the candidate and only using active/completed on real assignments. We'll update the helper default before we connect everything.

API errors will look like `{errorCode, error, requestId}`. Error codes: UNAUTHORIZED, VALIDATION_ERROR, SOURCE_UNAVAILABLE, FILE_TOO_LARGE, AI_TIMEOUT, DUPLICATE, NOT_FOUND. Error messages and logs should never include secrets, tokens or the student's imported text.

6.3.3.5 Brightspace Usage Log

One entry gets saved every time Gather calls the Brightspace API. LMU ITS asked for this.

{id, uid, action, method, endpoint, statusCode, success, durationMs, timestamp}

- uid - which student the call was for
- action - what we were doing: token_exchange, token_refresh, token_revoke, fetch_enrollments or fetch_assignments
- method / endpoint - the HTTP method and the Brightspace path we called, without the query string
- statusCode - the HTTP status Brightspace sent back (0 if the request never got a response, like a timeout)
- success - true if the status was 2xx
- durationMs - how long the call took
- timestamp - server timestamp

We don't save tokens, authorization codes, client secrets, or anything Brightspace sends back (like assignment names) in the log. It's just enough to show when and how we used the API.

6.3.3.6 Duplicate Detection

If there's no Brightspace ID to compare, we make a key by hashing (SHA-256) the user ID, source, course ID, title and date after normalizing them. If Brightspace gives us an ID we save that too. Before approving, we check against both existing tasks and pending candidates. If an approval gets retried (like after a timeout) it should return the original assignment ID instead of making a new one.

6.3.4 Detailed Design Diagrams

These diagrams go into more detail than the ones in 6.2.3: a detailed use case diagram, a package diagram, a class diagram, two sequence diagrams, an activity diagram and a state diagram.

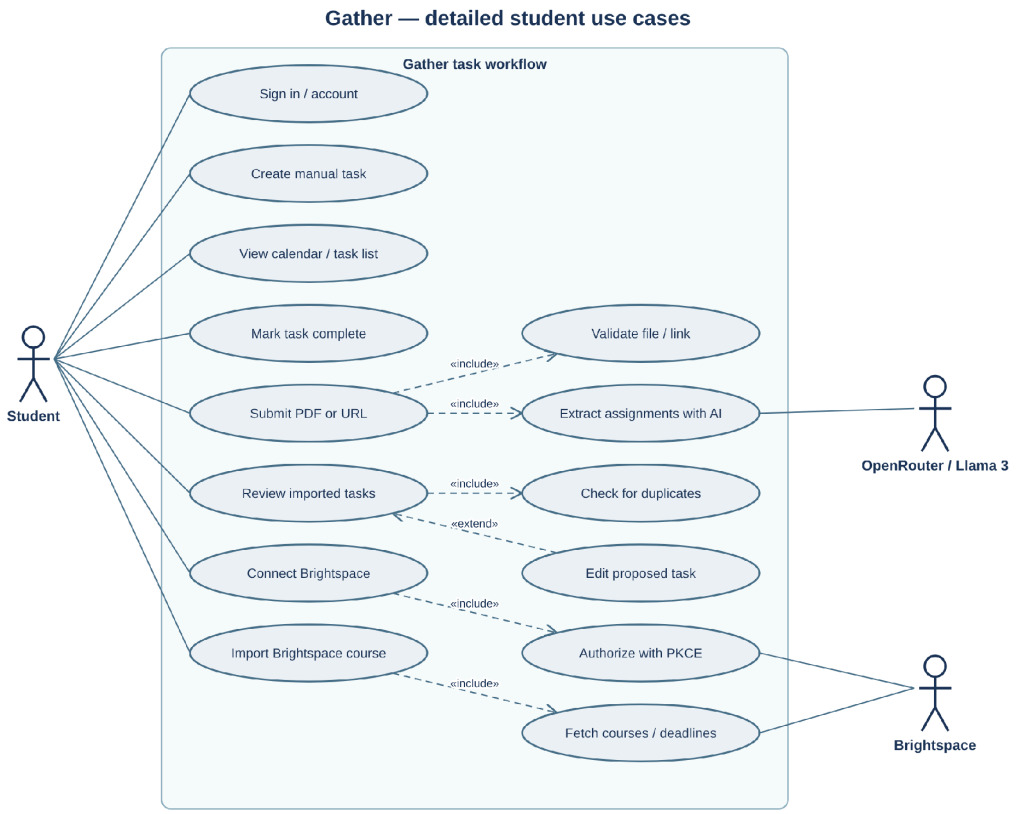


Figure 6-5. Detailed use case diagram

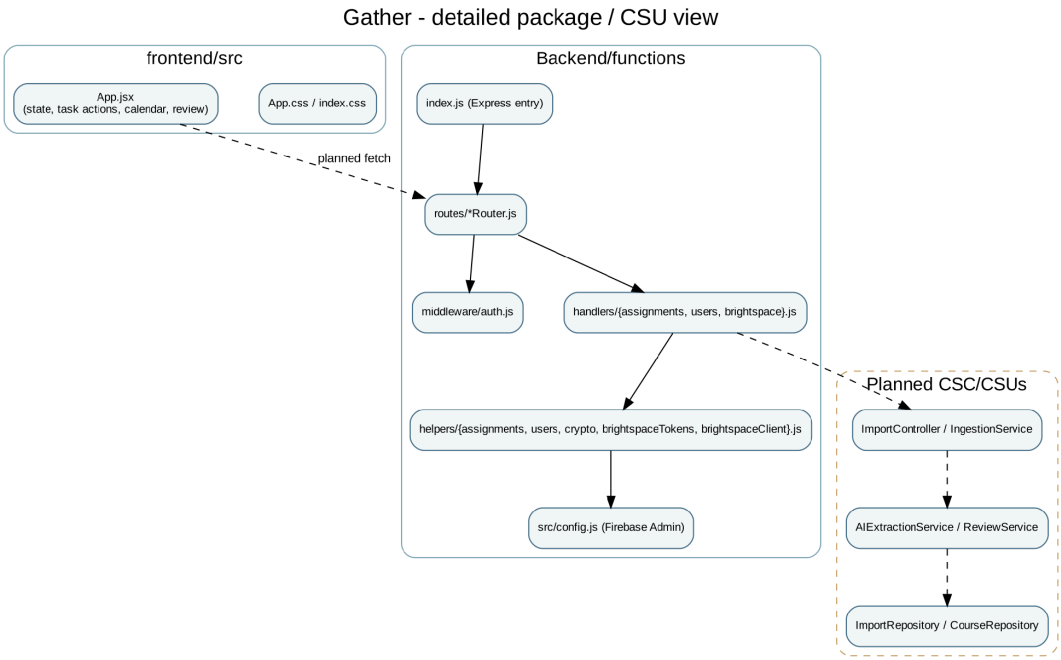


Figure 6-6. Package diagram (current files and planned parts)

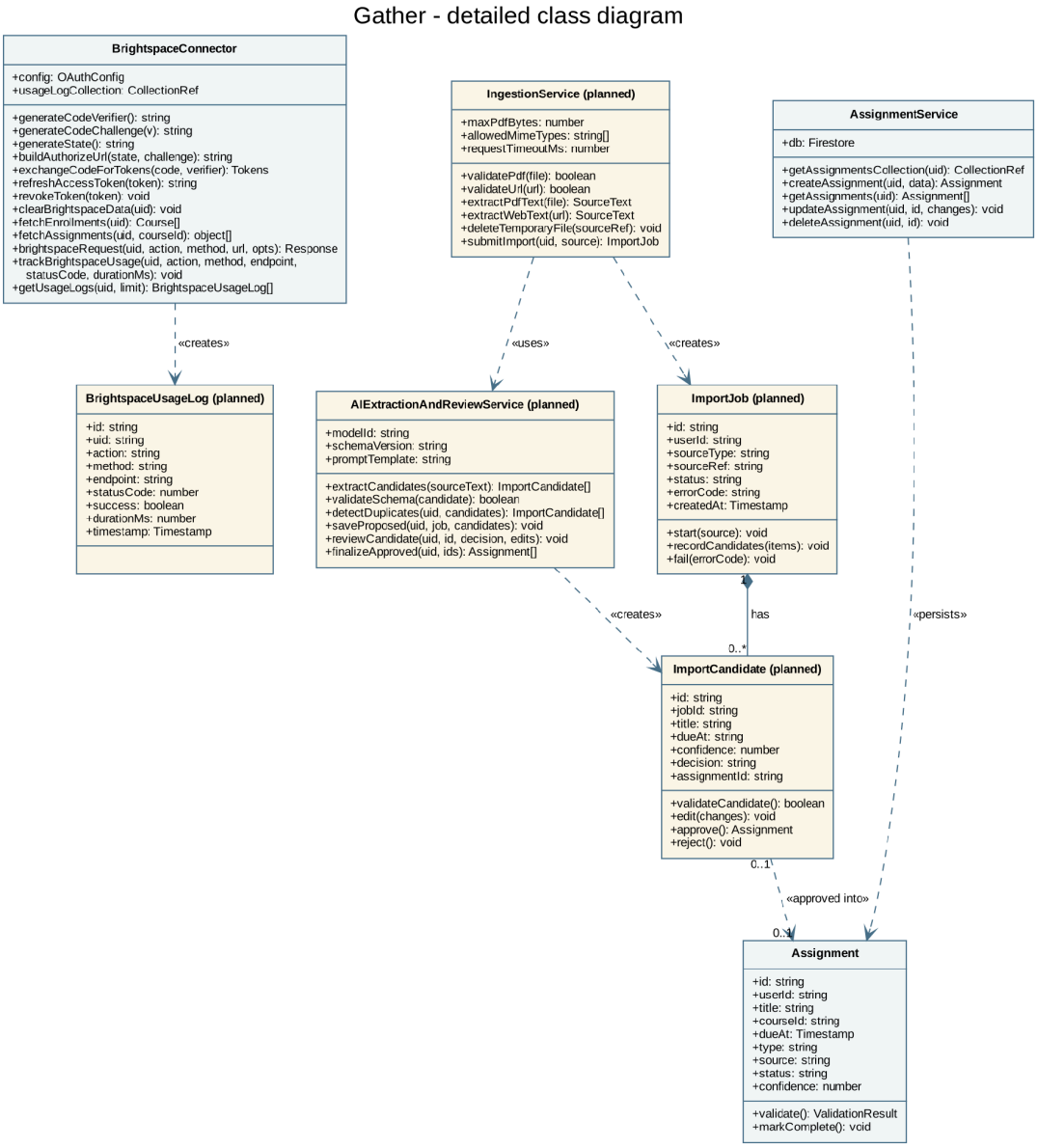


Figure 6-7. Detailed class diagram

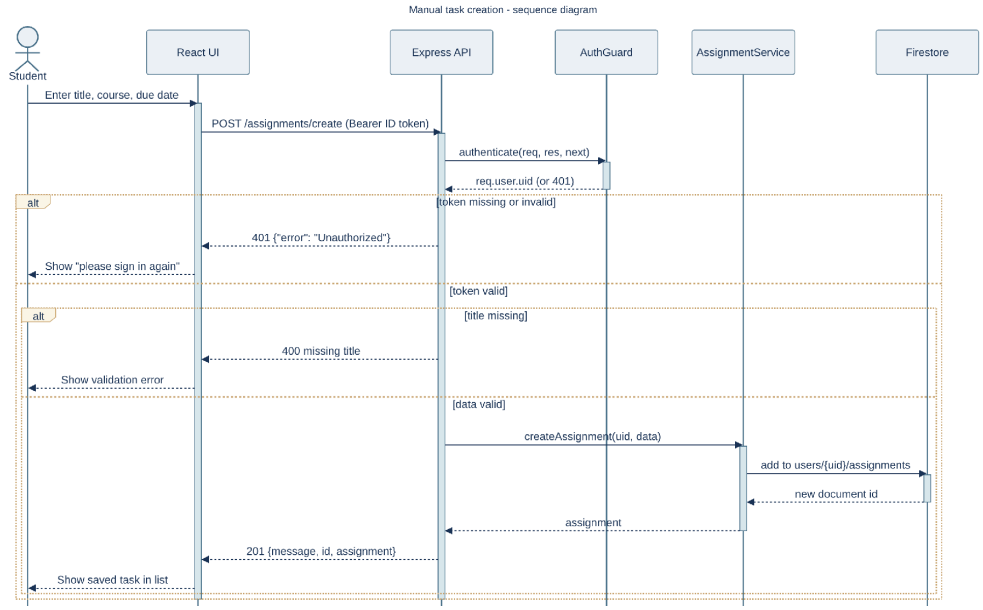


Figure 6-8. Sequence diagram - creating a task manually (with login check)

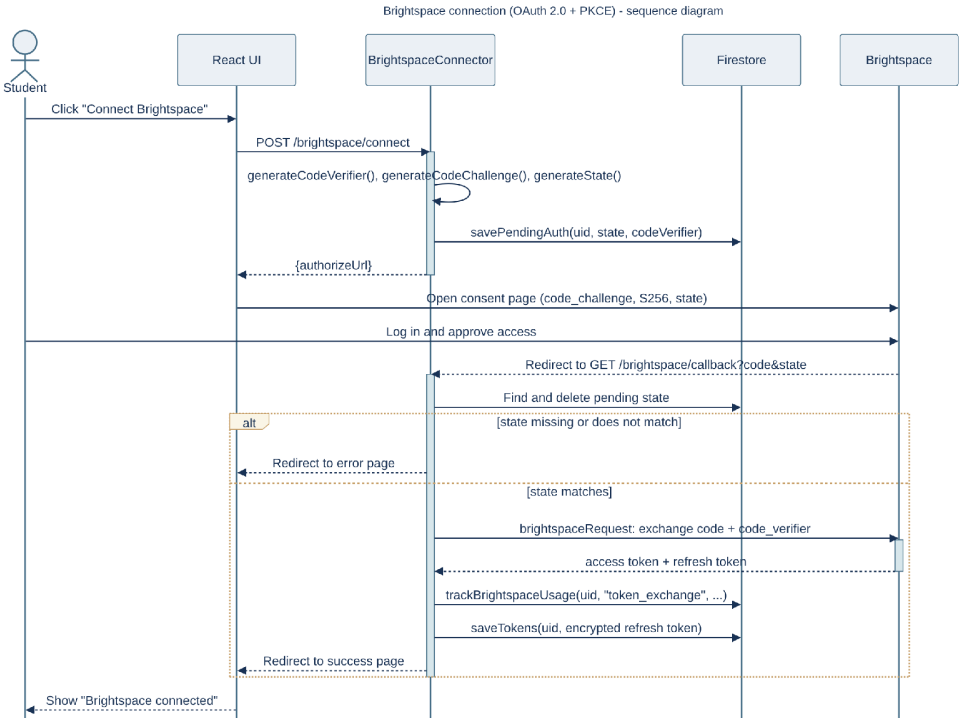


Figure 6-9. Sequence diagram - connecting Brightspace with PKCE (with usage logging)

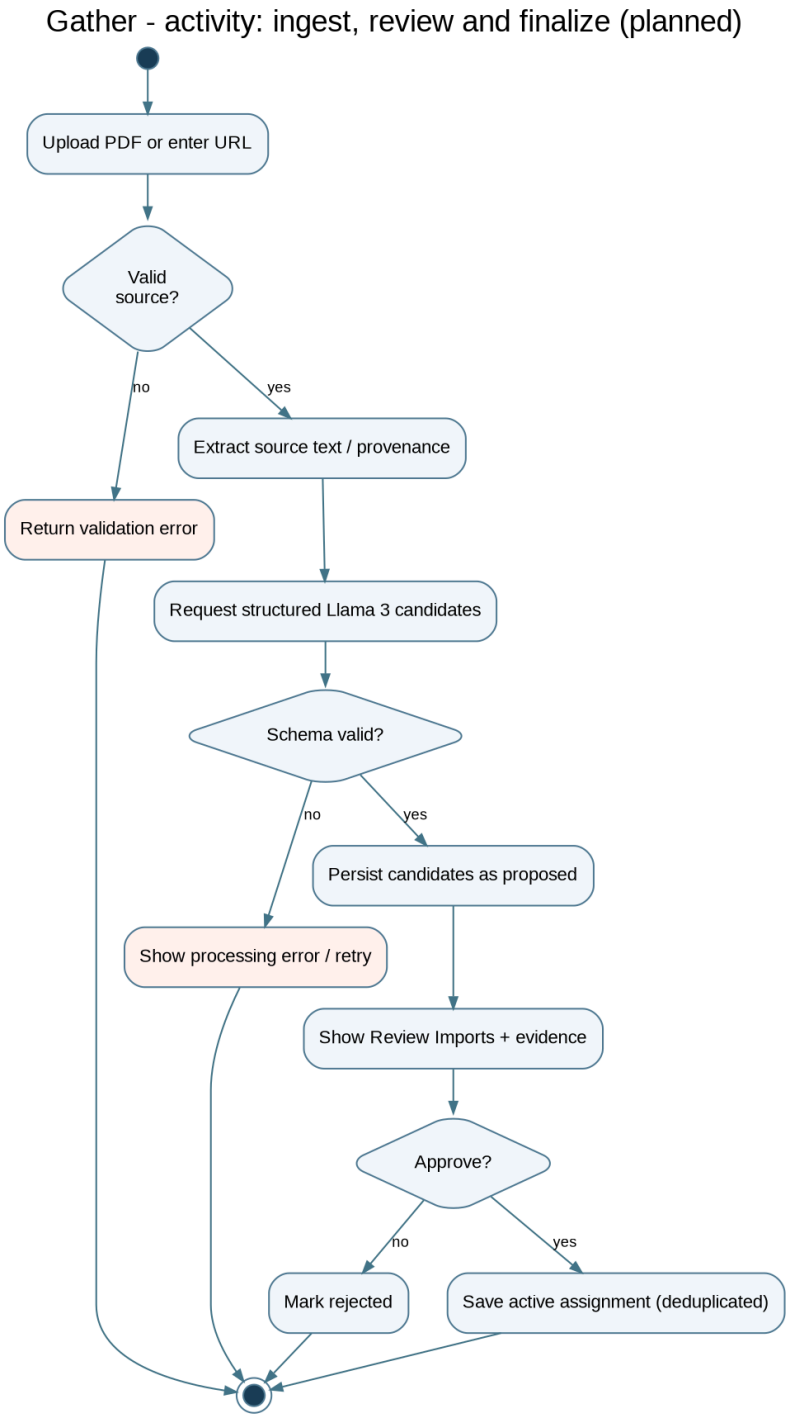


Figure 6-10. Activity diagram - importing a PDF/URL and reviewing it

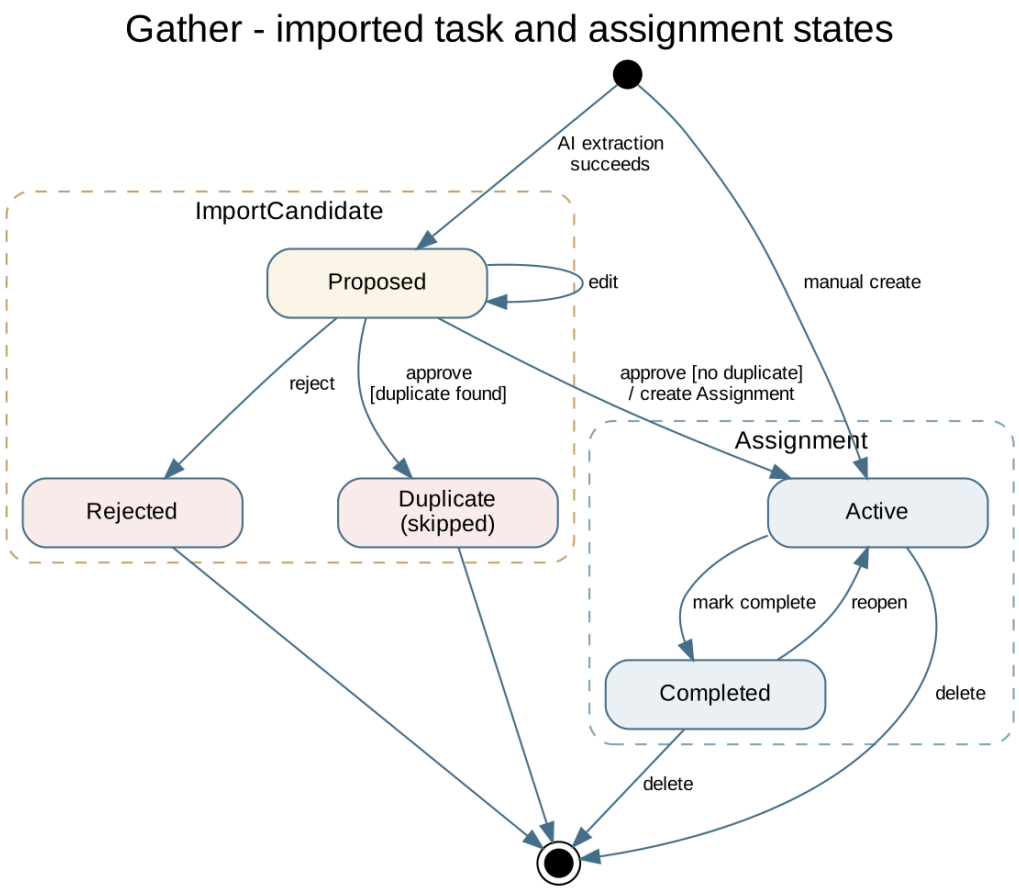


Figure 6-11. State diagram - imported task and assignment states

6.4 Database Design and Description

We're using Google Cloud Firestore, which is a NoSQL document database. Right now we have a top level users collection (and a planned top level brightspaceUsageLogs collection), and under each user there's an assignments collection and a brightspaceAuth collection. The SRS also needs saved classes, proposed AI assignments and temporary PDF files, so those are in the diagram as planned (the yellow boxes).

6.4.1 Database Design ER Diagram

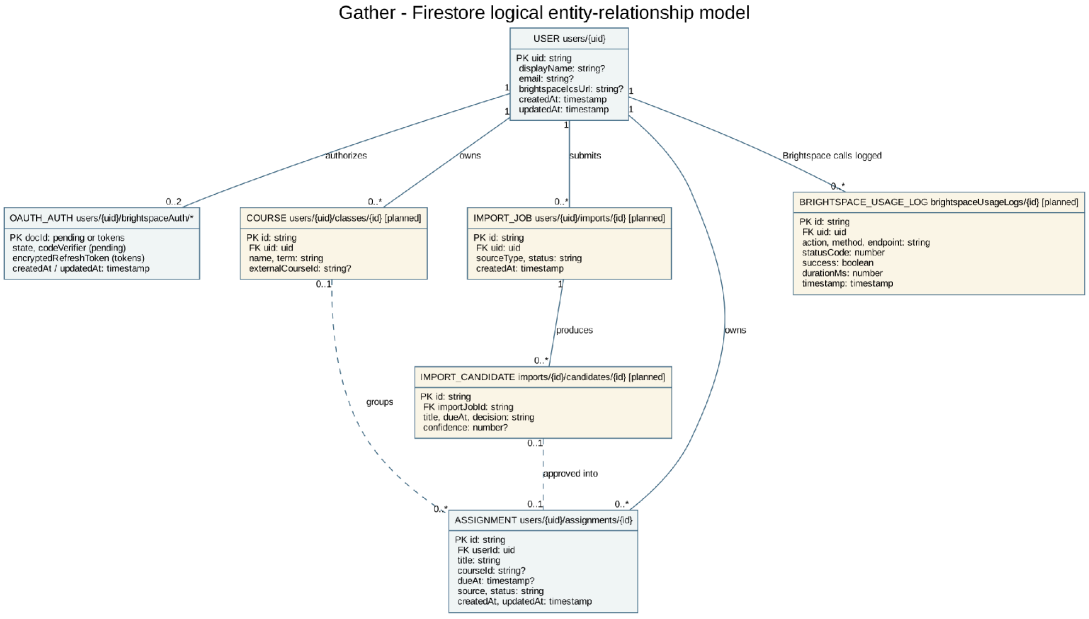


Figure 6-12. ER diagram (yellow = planned)

Firestore doesn't really have foreign keys like SQL, it uses nested paths instead, so this diagram shows how things are related logically. Each assignment belongs to one user, and a user can have many assignments and many import jobs. Each import job can make many candidates, and an approved candidate makes one assignment. Course is optional because manual tasks don't have to be linked to a class. A user can have many Brightspace usage logs. We put the usage logs in a top level collection (instead of under each user) so it's easy to look through all Brightspace usage at once if ITS asks.

The actual paths are:

- users/{uid}
- users/{uid}/assignments/{assignmentId}
- users/{uid}/brightspaceAuth/pending
- users/{uid}/brightspaceAuth/tokens
- brightspaceUsageLogs/{logId} (planned)
- users/{uid}/classes/{courseId} (planned)
- users/{uid}/imports/{jobId}/candidates/{candidateId} (planned)

Uploaded PDFs go in Cloud Storage temporarily, not in Firestore.

6.4.2 Database Access

The Express Cloud Function uses `admin.firestore()` to read and write. AssignmentService always gets the collection using the uid from the verified token (`getAssignmentsCollection(uid)`), so one user can't get to another user's data. UserService uses users/{uid}, the Brightspace token helpers use users/{uid}/brightspaceAuth, and trackBrightspaceUsage() adds new documents to brightspaceUsageLogs. Usage logs are only ever added, never edited. To show a user's logs we query brightspaceUsageLogs by uid and sort by timestamp, which will need a composite index (uid + timestamp).

For create, the server checks required fields and adds createdAt/updatedAt. For read, it gets the user's assignments and adds the document IDs to the JSON. For update, only certain fields should be allowed to change. Right now the handler removes userId but we still need to add a full list of allowed fields. For delete, it deletes by ID under the user's own collection. When we build the review part, approving a candidate and creating the assignment should happen in one Firestore transaction or batch so they don't get out of sync.

firestore.indexes.json doesn't have any custom indexes right now. We'll add them once we know what queries we actually need (probably assignments by status + dueAt, and candidates by decision). We'll also limit/paginate results per user. The speed targets from SRS 5.4 will need to be tested once everything is connected.

6.4.3 Database Security

The goal is that each student can only see their own profile, assignments, classes and imports. If the browser ever talks to Firestore directly, the rules need to check `request.auth != null && request.auth.uid == uid`. The Brightspace tokens and the usage logs should never be readable or writable from the browser, only from the backend. On the backend every route checks the Firebase ID token and uses the uid from the token, never a uid sent in the request body.

Right now Backend/firestore.rules is still the default test mode rules that allow anyone to read and write until November 9, 2026. That doesn't meet SRS 5.3.3.2, so we need to replace it with real per user rules before we release. The Admin SDK ignores these rules anyway, so the backend has to do its own checks too.

Brightspace refresh tokens are encrypted with AES-256-GCM using Node crypto and a 256 bit key from TOKEN_ENCRYPTION_KEY. The IV, auth tag and ciphertext are stored together. We should keep that key in a secret manager, limit who can access the service account, and never put raw tokens in anything the client can see. The pending OAuth state gets deleted when it's used, but we still need to add a check so old state values expire.

Other things we're doing for privacy: checking uploaded PDFs, using HTTPS everywhere, deleting uploaded files after extracting the text, not sending any secrets to OpenRouter, and not keeping more of the student's content than we need. We still need to decide on how long to keep data and how to back it up. For the Brightspace usage logs we'll ask LMU ITS how long they want us to keep them.